# Experiment-5

# Missionaries Cannibal Problem

**Aim:**

To develop a Python program to solve the Missionaries and Cannibals Problem using the Breadth-First Search (BFS) algorithm.

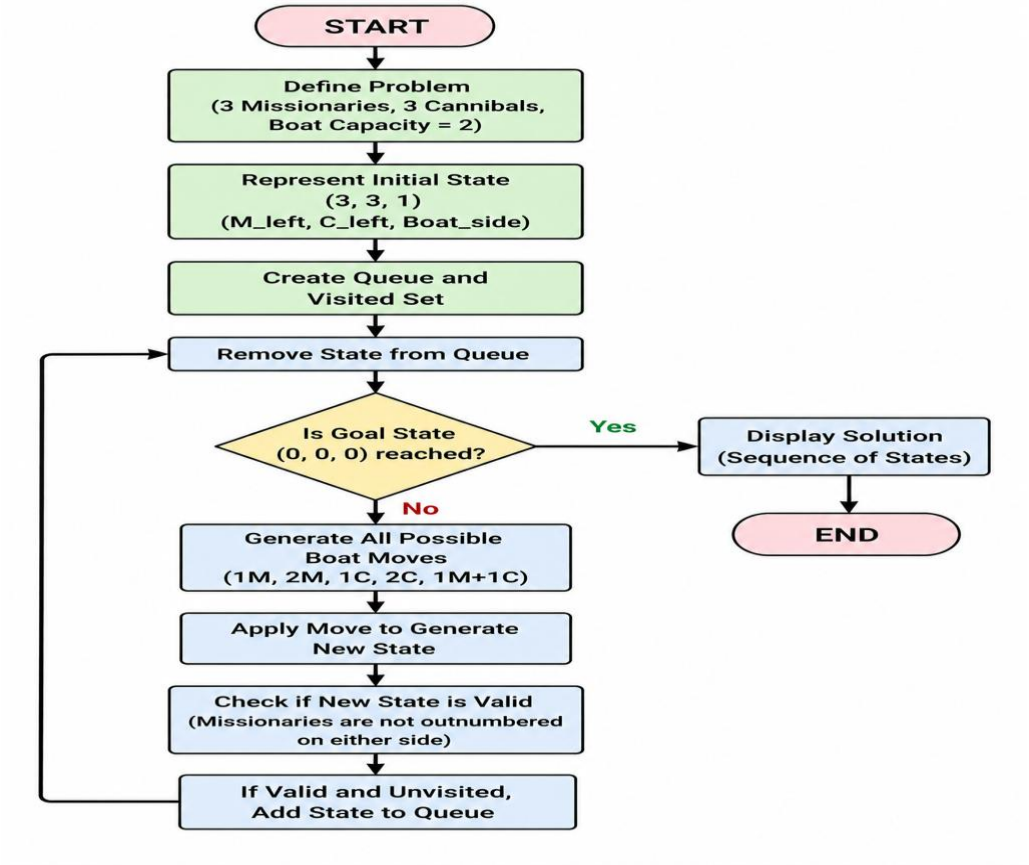
**Objective:**

- To solve the Missionaries and Cannibals problem using BFS.
- To safely transport all missionaries and cannibals across the river.
- To understand state-space search techniques in Artificial Intelligence.

**Algorithm:**

1. Start the program.
2. Represent the initial state as **(3,3,1)**.
3. Create a queue and a visited set.
4. Remove a state from the queue.
5. If the goal state **(0,0,0)** is reached, display the solution.
6. Generate all valid boat moves.
7. Check whether each new state is safe.
8. Add unvisited valid states to the queue.
9. Repeat until the goal state is found.
10. Stop the program.

**Flowchart:**




**Code:**

```
from collections import deque


def valid(m, c):
    return 0 <= m <= 3 and 0 <= c <= 3 and (m == 0 or m >= c) and (3-m == 0 or 3-m >= 3-c)


def bfs():
    q = deque([((3,3,1), [])])
    v = set()
    while q:
        (m,c,b), path = q.popleft()
        if (m,c,b) in v:
            continue
        v.add((m,c,b))
```

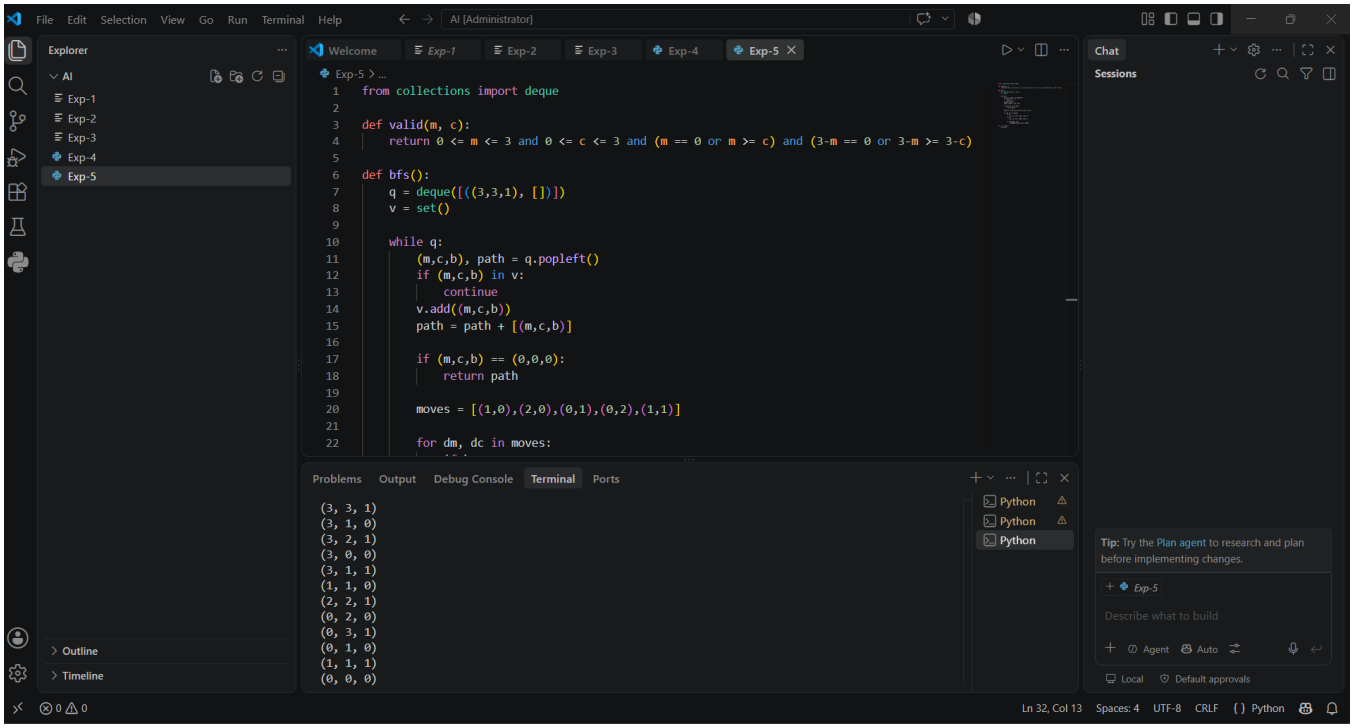
```
        path = path + [(m,c,b)]
        if (m,c,b) == (0,0,0):
            return path
        moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]

        for dm, dc in moves:
            if b:
                nm, nc, nb = m-dm, c-dc, 0
            else:
                nm, nc, nb = m+dm, c+dc, 1

            if valid(nm, nc):
                q.append(((nm,nc,nb), path))

for s in bfs():
    print(s)
```

**Output:**



**Result:**

The Missionaries and Cannibals Problem was successfully solved using the Breadth-First Search (BFS) algorithm.

**Conclusion:**

The Breadth-First Search (BFS) algorithm efficiently found a safe sequence of moves to transport all missionaries and cannibals across the river.